

Práctica 1



Jairo Moraga Monterroso – jairo.moraga@edu.uah.es

Luis Miguel Herrá – luis.herra@edu.uah.es

Planificación Automática

Grado en Ingeniería informática

Introducción

Primera Parte

Ejercicio 1.1: Modelado del dominio de logística de emergencias en PDDL, versión inicial.

Descripción general del dominio

En este ejercicio se ha modelado un **dominio de planificación** clásica en **PDDL** (nivel STRIPS con :typing) que representa un sistema de **reparto de suministros** de emergencia mediante un **dron**.

El objetivo del dominio es permitir que un dron transporte cajas desde un depósito hasta distintas localizaciones con personas heridas, de forma que estas reciban los contenidos que necesitan (por ejemplo, comida o medicina).

El modelado se ha realizado **respetando** estrictamente las **restricciones indicadas** en el enunciado: uso exclusivo de STRIPS, uso de :typing, no utilizar precondiciones negativas, no utilizar metas negativas, no utilizar efectos condicionales, no incluir comentarios en los archivos PDDL.

Tipos definidos

Se han definido los siguientes tipos:

drone: representa los drones disponibles.

person: personas heridas.

crate: cajas transportables.

location: localizaciones del problema.

content: tipo de contenido de las cajas (comida, medicina, etc.).

arm*: brazos del dron.

Nota: La inclusión del tipo arm permite modelar correctamente la restricción de que el dron dispone de dos brazos y puede transportar como máximo dos cajas simultáneamente.

Modelado

Modelado de localización

Se han definido predicados independientes para representar la localización de cada entidad:

(at-drone ?d ?l), (at-person ?p ?l), (at-crate ?c ?l)

De esta forma se permite que múltiples personas estén en la misma localización, tal y como especifica el enunciado. El **dron ?d** está en la **ubicación ?l**; La **persona ?p** está en la ubicación ?l; La **caja ?c** está en la ubicación ?l.

Modelado genérico del contenido de las cajas

El contenido de las cajas se ha modelado mediante el predicado:

(crate-has ?c - crate ?t - content)

Esta decisión permite que el dominio sea completamente genérico respecto al tipo de contenidos. Si se desea añadir nuevos tipos (p.e agua, mantas, etc.), basta con añadir nuevos objetos de tipo content en el problema, sin necesidad de modificar el dominio.

Esto cumple explícitamente la restricción del enunciado de no utilizar predicados específicos como (caja-comida ?c) o (caja-medicina ?c).

Modelado de los brazos del dron

Para modelar que el dron dispone de dos brazos y puede transportar un máximo de dos cajas, se han definido:

(arm-of ?a ?d) → indica a qué dron pertenece cada brazo.

(free ?a) → indica si el brazo está libre.

(holding ?a ?c) → indica que un brazo está sujetando una caja.

Este diseño permite: **limitar el número de cajas** transportadas simultáneamente; **facilitar** la futura **extensión** del dominio a múltiples drones; **evitar** el uso de **contadores** o **extensiones** no permitidas en STRIPS.

Modelado de las acciones

La **Acción fly** permite que el dron se desplace entre cualquier par de localizaciones. Precondición: el dron debe estar en la localización origen. Efectos: elimina la localización anterior y se añade la nueva localización. No modelar rutas ni conexiones específicas; el enunciado indica que el dron puede volar directamente entre cualquier par de localizaciones.

La **Acción pick-up** permite que el dron recoja una caja utilizando uno de sus brazos. Precondiciones: El dron y la caja están en la misma localización, el brazo pertenece al dron y el brazo está libre. Asimismo, sus efectos son: la caja deja de estar en la localización, el brazo deja de estar libre, y el brazo pasa a sostener la caja.

La **Acción deliver** permite entregar una caja a una persona concreta. Precondiciones: El dron y la persona están en la misma localización, el brazo pertenece al dron, el brazo está sosteniendo la caja y la caja contiene el tipo de contenido correspondiente. Por otro lado

sus Efectos serían que la persona pasa a tener ese contenido, que el brazo queda libre y que se eliminaría la relación de que el brazo sostiene la caja.

El modelo no registra qué caja recibe cada persona, sino únicamente el contenido que obtiene, ya que el enunciado indica que lo relevante es el tipo de contenido y no la identidad concreta de la caja.

Problemas generados

Se han construido dos instancias:

Problema 1: 1 persona, 1 caja y 1 tipo de contenido.

Objetivo: que la persona reciba dicho contenido.

Problema 2: 2 personas, 3 cajas, 2 tipos de contenido.

Cada persona requiere un contenido distinto.

En ambos casos todas las cajas comienzan en el depósito, el dron comienza en el depósito y no hay personas en el depósito. Ambos problemas se han resuelto con el planificador FF.

Consideraciones de diseño

Las principales decisiones de modelado han sido:

Uso de un modelo completamente genérico para el contenido.

Modelado explícito de los brazos para respetar la restricción física del dron.

Separación clara entre localización de personas, cajas y dron.

Diseño preparado para futura extensión a múltiples drones sin necesidad de modificar la estructura del dominio.

Gracias a estas decisiones tomadas nuestro dominio puede ser escalable en el futuro y es compatible con planificadores clásicos como FF y pyperplan.

Ejercicio 1.2: Generador de problemas en Python

Explicación del generador de problemas en Python realizado para este ejercicio.

Para poder evaluar el rendimiento del planificador de forma sistemática, se desarrolló un generador automático de instancias en Python. Este generador permite crear problemas de diferente tamaño a partir de una serie de parámetros que especifican el número de drones, localizaciones, personas, cajas y metas.

El programa recibe estos parámetros mediante argumentos de línea de comandos y construye dinámicamente todos los objetos necesarios del dominio. En primer lugar, genera las listas de drones, personas, cajas y localizaciones, incluyendo siempre una localización especial denominada depot. Asimismo, para cada dron se crean automáticamente dos objetos de tipo arm, que representan sus brazos, garantizando así el cumplimiento de la restricción física del dominio.

Posteriormente, el generador asigna contenidos a las cajas. Para ello, distribuye aleatoriamente las cajas entre los distintos tipos de contenido definidos (por ejemplo, food y medicine), asegurando que exista al menos una caja de cada tipo y que el número total de

metas solicitadas sea alcanzable con los recursos disponibles. Esta comprobación evita generar problemas irresolubles.

A continuación, se generan las necesidades de las personas, seleccionando aleatoriamente qué personas requieren qué tipo de contenido, respetando siempre el límite de cajas disponibles para cada tipo. Este procedimiento garantiza la coherencia entre el estado inicial y el objetivo del problema.

En el estado inicial, todos los drones y cajas se sitúan en el depósito, mientras que las personas se distribuyen aleatoriamente entre las localizaciones distintas del depósito. También se inicializan correctamente las relaciones que indican que cada brazo pertenece a un dron y que comienza libre.

Finalmente, el programa construye la sección de objetivos del problema. En ella se incluyen los hechos correspondientes a las necesidades generadas para las personas y, adicionalmente, se puede imponer que los drones finalicen en el depósito. El resultado es un fichero PDDL completamente válido y compatible con el dominio definido previamente.

Este generador permite crear de forma automática secuencias de problemas de tamaño creciente manteniendo una estructura coherente, lo que resulta fundamental para realizar experimentos controlados de evaluación del rendimiento del planificador.

Experimentos con el planificador FF

Para evaluar el rendimiento del planificador FF se generaron problemas de complejidad creciente manteniendo:

$$d = 1, r = 0 \text{ y } l = p = c = g$$

El tamaño del problema se incrementó progresivamente desde 5 hasta 55.

Los tiempos obtenidos fueron los siguientes:

Tamaño	Tiempo (s)
5	0.00
10	0.00
20	0.23
30	3.18
35	6.12
40	10.95
41	12.98
42	35.07
43	32.64
44	56.31
45	31.01
46	46.46

47	100.72
48	101.51
49	88.79
50	64.70
51	260.63
52	109.04
55	498.76

Conclusiones

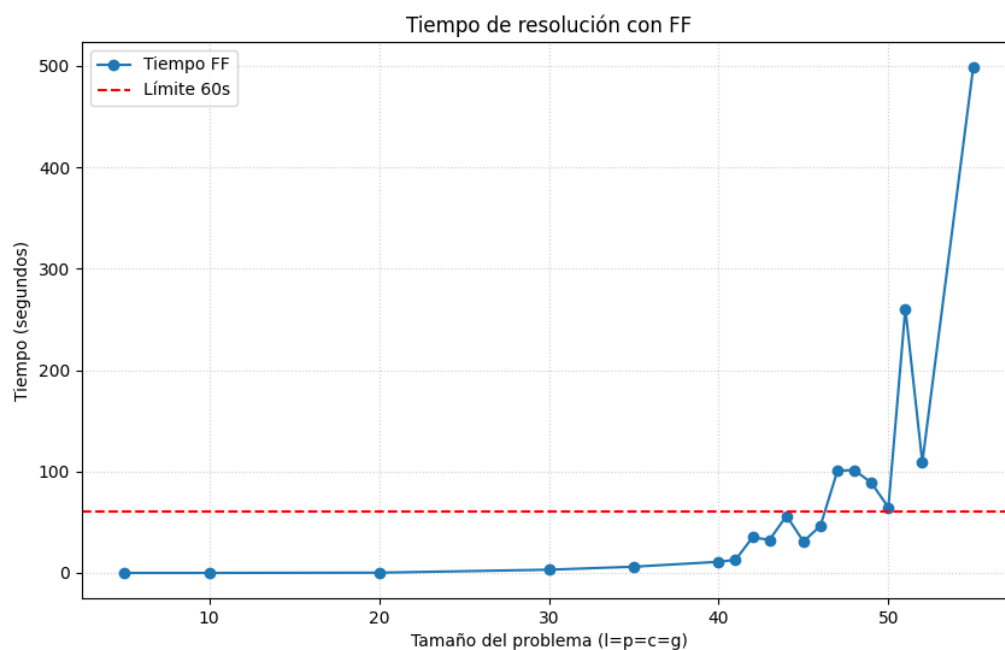
Se observa que el tiempo de resolución crece de manera claramente no lineal. A partir de tamaños cercanos a 40 comienza una explosión combinatoria que incrementa significativamente el tiempo de búsqueda.

Aunque los tiempos no crecen de forma estrictamente monótona debido a la aleatoriedad en la generación de instancias, el comportamiento general muestra un crecimiento cercano a exponencial.

Considerando un límite de 1 minuto, el mayor tamaño de problema que puede resolverse consistentemente dentro del tiempo límite es aproximadamente entre **44 y 46 objetos por parámetro**.

Gráfica Comparativa

A continuación, se presenta la gráfica cruzando el tamaño del problema y el tiempo requerido en para encontrar sus respectivas soluciones en las pruebas realizadas. La gráfica se ha hecho con un programa Python que estará disponible dentro de la carpeta de esta primera parte de la Práctica 1.



Análisis del comportamiento del planificador FF según los resultados gráficos.

El planificador FF es un planificador heurístico de tipo satisficing, es decir, está diseñado para encontrar soluciones de manera eficiente sin garantizar que estas sean óptimas; es un algoritmo voraz. A diferencia de planificadores óptimos como A*, FF prioriza la rapidez en la obtención de un plan válido frente a la minimización del número de acciones. Esto explica su excelente rendimiento en problemas pequeños y medianos, donde es capaz de encontrar soluciones prácticamente instantáneas.

El núcleo del funcionamiento de FF se basa en una heurística relajada. En concreto, construye una versión relajada del problema eliminando los efectos negativos de las acciones (ignore delete list). Esto significa que, al estimar la distancia al objetivo, el planificador asume que los hechos nunca se eliminan del estado. Esta relajación simplifica enormemente el problema y permite calcular rápidamente una estimación heurística informativa. Sin embargo, **esta aproximación puede perder precisión en dominios donde los efectos negativos son relevantes**, como ocurre en este dominio cuando un brazo deja de estar libre o cuando una caja deja de estar en una localización.

En problemas de tamaño reducido, esta heurística es muy efectiva, ya que la estructura del dominio es relativamente simple y la estimación relajada se aproxima bastante bien al coste real. Por ello, se observa que hasta tamaños intermedios el tiempo de resolución es prácticamente nulo. No obstante, a medida que aumenta el número de localizaciones, personas y cajas, el espacio de estados crece de manera muy rápida, lo que provoca un incremento significativo del tiempo de búsqueda.

El crecimiento del espacio de estados en este dominio es particularmente acusado debido a varios factores combinatorios. En primer lugar, cada caja puede encontrarse en distintas localizaciones o estar siendo transportada por alguno de los brazos del dron, lo que multiplica el número de configuraciones posibles. En segundo lugar, el dron puede situarse en cualquier localización, lo que añade otra dimensión al espacio de búsqueda. Además, el hecho de que el dron disponga de dos brazos introduce múltiples combinaciones posibles de transporte simultáneo de cajas.

A esto se añade el componente combinatorio del reparto: cada persona puede necesitar distintos tipos de contenido y existen múltiples formas válidas de satisfacer las metas, ya que no importa qué caja concreta se entregue, sino únicamente su contenido. Esta flexibilidad incrementa el número de planes potenciales y, por tanto, el número de estados intermedios que el planificador puede explorar.

Como consecuencia de todos estos factores, el espacio de búsqueda crece de forma aproximadamente exponencial con el tamaño del problema. Esto explica el incremento observado en los tiempos de resolución a partir de cierto umbral (en torno a tamaños cercanos a 40–45), donde pequeñas variaciones en el número de objetos generan incrementos muy significativos en el tiempo necesario para encontrar una solución.

Ejercicio 1.3: Comparativa rendimiento algoritmos de búsqueda y heurísticas

Punto 1: Comparación de distintos algoritmos

Algoritmo	Tamaño Máximo	Tiempo (s)	Longitud Plan	Óptimo
BFS	7	28.008s	22	Sí
IDS	3	0.713s	10	Sí
A* + hMAX	5	10.196s	17	Sí
GBFS + hMAX	8	52.802s	30	No

Los resultados obtenidos muestran diferencias significativas en el rendimiento de los distintos algoritmos de búsqueda aplicados al dominio. Estas diferencias pueden explicarse tanto por las características del dominio como por la naturaleza de cada algoritmo.

En primer lugar, se observa que los **algoritmos de búsqueda no informada** (BFS e IDS) alcanzan **tamaños máximos** de problema considerablemente **menores** que los algoritmos informados. En concreto, IDS únicamente es capaz de resolver instancias de tamaño 3 dentro del límite temporal, mientras que BFS alcanza tamaño 7. Esto se debe a que ambos algoritmos **exploran** el espacio de estados **sin** utilizar información **heurística** que guíe la búsqueda hacia el objetivo. En un dominio como el presente, donde el número de combinaciones posibles crece rápidamente con el número de cajas, localizaciones y personas, el espacio de búsqueda se expande de manera exponencial. Como consecuencia, la exploración exhaustiva de niveles sucesivos (en BFS) o de profundidades crecientes (en IDS) se vuelve rápidamente inabordable.

Resulta especialmente llamativo el bajo rendimiento de IDS en comparación con BFS. Aunque ambos garantizan resultados óptimos, **IDS repite múltiples exploraciones** desde la raíz del árbol en cada incremento de profundidad, lo que introduce una sobrecarga considerable. En dominios con un factor de ramificación elevado, como ocurre aquí debido a las múltiples acciones posibles (volar entre muchas localizaciones, recoger distintas cajas, entregar a diferentes personas), este coste adicional penaliza fuertemente su rendimiento.

En cuanto a los algoritmos informados, se observa que A* con heurística hMAX mejora el rendimiento respecto a IDS, pero no supera a BFS en tamaño máximo alcanzado. Esto puede parecer ilógico, ya que A* utiliza información heurística. Sin embargo, la **heurística hMAX**, aunque es admisible y garantiza resultados óptimos, tiende a ser **poco informativa** en **dominios con múltiples objetivos** independientes. Al calcular el máximo coste individual de los subobjetivos ignorando efectos negativos, puede **infraestimar significativamente** el **coste real** cuando se requieren múltiples entregas. Esto provoca que A* siga expandiendo un número elevado de nodos, limitando su escalabilidad.

El algoritmo que alcanza **mayor tamaño** es **GBFS con hMAX**. Esto es coherente con su naturaleza satisficing: prioriza expandir el nodo que parece más prometedor según la heurística, sin preocuparse por garantizar optimalidad. Al no mantener una frontera

ordenada por coste acumulado como A*, reduce considerablemente el número de nodos explorados y puede avanzar más profundamente en el espacio de búsqueda. Sin embargo, el **plan generado no está garantizado como óptimo**, lo que se refleja en la mayor longitud del plan obtenido en comparación con BFS para tamaños inferiores.

En conjunto, los resultados ilustran claramente el trade-off entre optimalidad y escalabilidad. Los algoritmos óptimos (BFS, IDS y A*) garantizan soluciones mínimas pero escalan peor en dominios con crecimiento combinatorio elevado. Por el contrario, GBFS sacrifica optimalidad a cambio de una mayor capacidad para resolver instancias de mayor tamaño dentro del límite temporal.

El comportamiento observado está directamente relacionado con las características del dominio. Cada incremento en el número de localizaciones, personas y cajas multiplica el número de configuraciones posibles: el dron puede situarse en distintas localizaciones, cada caja puede estar en diferentes estados (en el depósito, en transporte o entregada), y las combinaciones de reparto crecen rápidamente. Como hemos explicado, este incremento combinatorio penaliza especialmente a los algoritmos que exploran el espacio de forma sistemática sin una guía heurística fuerte.

Punto 2: Heurísticas para planificadores satisficing

Algoritmo	Heurística	Tiempo (s)	Longitud Plan
GBFS	hMAX	52.802s	30
GBFS	hADD	0.375s	31
GBFS	hFF	0.467s	31
GBFS	Landmark	0.179s	32
EHC	hMAX	+1h	Killed (no se ha llegado a completar)
EHC	hADD	1.236s	30
EHC	hFF	0.892s	28
EHC	Landmark	0.526s	38

En este apartado se ha evaluado el comportamiento de los algoritmos GBFS y EHC combinados con distintas heurísticas (hMAX, hADD, hFF y Landmark) sobre el mayor problema que GBFS era capaz de resolver en menos de un minuto (**tamaño 8**). Los resultados muestran **diferencias muy significativas** tanto en **tiempo** de ejecución como en la **longitud** de los planes generados. En general, se observa que la elección de la heurística tiene un impacto mucho mayor en el rendimiento que la elección del algoritmo de búsqueda, especialmente cuando se emplea una heurística poco informativa como hMAX.

La **heurística hMAX** presenta el **peor rendimiento** en ambos algoritmos. Con GBFS, el tiempo de ejecución es elevado (52.802s), mientras que con EHC el proceso no llega a

completarse y es abortado. Este comportamiento se explica porque hMAX calcula el coste máximo individual de los subobjetivos ignorando efectos negativos, lo que produce estimaciones muy conservadoras y poco informativas. En un dominio como el presente, donde múltiples entregas deben realizarse y los subobjetivos interactúan indirectamente, esta heurística **no proporciona suficiente orientación**, obligando a los algoritmos a explorar un número elevado de estados.

Por el contrario, **hADD y hFF** muestran un comportamiento mucho **más eficiente**. Ambas heurísticas son más informativas, ya que consideran de manera más precisa el esfuerzo conjunto necesario para alcanzar múltiples metas. En particular, hADD suma los costes estimados de los subobjetivos, mientras que hFF construye un plan relajado completo para estimar la distancia al objetivo. Esto explica la drástica reducción en los tiempos de ejecución: en GBFS, los tiempos pasan de más de 50 segundos con hMAX a menos de medio segundo con hADD y hFF. En EHC ocurre algo similar, siendo especialmente destacable el buen rendimiento de hFF (0.892s) y hADD (1.236s). Esto confirma que en dominios logísticos con **múltiples metas** independientes, **heurísticas** que capturan la estructura **global** del problema son claramente superiores.

En cuanto a la **comparación** entre GBFS y EHC, las diferencias se explican por la naturaleza de cada algoritmo. **GBFS** mantiene una **frontera** de búsqueda y **expande** siempre el **nodo** con **mejor valor heurístico**, lo que le proporciona mayor robustez frente a mesetas o regiones planas del espacio de búsqueda. EHC, en cambio, implementa una estrategia de **búsqueda local**: intenta **avanzar** siempre hacia un estado con **mejor valor heurístico** y, si no lo encuentra, realiza una **búsqueda en anchura** limitada hasta hallar una mejor.

Esta estrategia puede ser extremadamente eficiente cuando la heurística es informativa (como ocurre con hFF), pero puede fallar o volverse muy costosa cuando la heurística es pobre, como se observa con hMAX. El hecho de que EHC con hMAX no complete la ejecución muestra su dependencia a la calidad de la heurística.

Finalmente, es importante destacar que el **planificador FF combina** precisamente **EHC** con la heurística **hFF** como estrategia principal. FF utiliza EHC para avanzar rápidamente guiado por la heurística del plan relajado y, en caso de fallo, puede recurrir a una búsqueda más sistemática. Los resultados obtenidos en este experimento respaldan esta elección de diseño: la combinación EHC + hFF **ofrece un equilibrio** excelente entre **rapidez** y **calidad** de la solución, produciendo planes más cortos que otras combinaciones y con tiempos de ejecución muy reducidos. En **conclusión**, los resultados experimentales reflejan claramente cómo la **calidad de la heurística** es **determinante** en **planificación** automática y **justifican** las decisiones de **diseño** adoptadas en **planificadores clásicos como FF**.

Punto 3: Heurísticas para planificadores óptimos

Investigamos qué heurísticas son admisibles en pyperplan

Una heurística es **admisible** SI nunca sobreestima el coste real mínimo hasta el objetivo.

Es decir:

$h(n) \leq h^*(n)$ para todo estado n .

En pyperplan:

hMAX → **Admisible**. Calcula el coste máximo individual de los subobjetivos en el problema relajado. No suma costes. Nunca sobreestima. Es monotónica (consistente).

hADD → **No admisible**. Suma los costes individuales de los subobjetivos. En problemas con subobjetivos que comparten acciones, puede contar acciones varias veces. Puede sobreestimar.

hSA → **No admisible**. Variante de heurística aditiva. También puede sobreestimar. No es admisible en general.

hFF → **No admisible**. Construye un plan relajado completo. Ese plan no tiene por qué ser el mínimo. Puede sobreestimar.

Landmark → **No admisible** (en general). Basada en contar hechos obligatorios (landmarks). No garantiza no sobreestimar. No está diseñada para optimalidad.

lmcut → **Admisible**. Es una heurística basada en cortes (landmark-cut). Está diseñada específicamente para ser admisible. Es consistente. Se usa en planificadores óptimos modernos.

Para este ejercicio, las heurísticas admisibles son hMAX y lmcut. Recordamos que el tamaño de problema a utilizar es 5 (mayor tamaño que puede resolver el algoritmo A* con heurística hMAX en un min)

Algoritmo	Tiempo (s)	Longitud Plan
BFS	0.583s	17
IDS	68m57.772s	Killed (no se ha llegado a completar)
A* + hMAX	8.672s	17
A* + lmcut	35.282s	17

Ahora se ha evaluado el rendimiento de los algoritmos óptimos sobre el mayor problema que A* con hMAX era capaz de resolver en menos de un minuto (tamaño 5). Se han comparado BFS, IDS y A* con las heurísticas admisibles hMAX y lmcut. En todos los casos en los que se obtuvo solución, la longitud del plan fue de 17 acciones, lo que confirma que los algoritmos utilizados (BFS y A* con heurísticas admisibles) garantizan optimalidad y que el coste óptimo del problema es efectivamente 17.

El algoritmo BFS resolvió el problema en 0.583 segundos, mostrando un rendimiento sorprendentemente bueno en comparación con A*. Esto se explica porque, para tamaños relativamente pequeños, el coste de mantener estructuras de datos complejas y calcular heurísticas puede superar el beneficio de la guía heurística. BFS realiza una exploración por niveles y, dado que la profundidad óptima del plan no es excesivamente grande, logra alcanzar la solución sin necesidad de explorar una cantidad inasumible de nodos.

Por el contrario, IDS no logró completar la ejecución dentro de un tiempo razonable, siendo finalmente abortado tras un periodo prolongado. Este comportamiento es coherente con la naturaleza del algoritmo: IDS repite la exploración desde la raíz para cada incremento de profundidad, lo que genera una gran redundancia en dominios con alto factor de ramificación. En este dominio, cada estado ofrece múltiples acciones posibles (volar a distintas localizaciones, recoger distintas cajas, realizar entregas), lo que multiplica el número de nodos explorados en cada iteración y hace que la repetición inherente a IDS resulte extremadamente costosa.

En cuanto a A*, se observa que con hMAX el tiempo de ejecución es de 8.672 segundos, mientras que con lmcut asciende a 35.282 segundos. Aunque ambas heurísticas son admisibles y producen planes óptimos, su comportamiento práctico difiere. La heurística hMAX es menos informativa, ya que solo considera el coste máximo individual de los subobjetivos en el problema relajado, lo que puede conducir a una exploración relativamente amplia del espacio de estados. Por su parte, lmcut es una heurística más sofisticada y generalmente más informativa, pero su cálculo es computacionalmente más costoso. En problemas de tamaño reducido, el coste adicional de computar lmcut puede superar el ahorro en nodos explorados, lo que explica que en este caso concreto resulte más lenta que hMAX.

En conjunto, los resultados reflejan claramente el compromiso entre coste de evaluación heurística y reducción del espacio de búsqueda. En problemas pequeños, algoritmos simples como BFS pueden competir favorablemente con métodos heurísticos. Sin embargo, a medida que el tamaño del problema crece, el uso de heurísticas informativas se vuelve esencial para mantener la escalabilidad. Además, la ineficiencia observada en IDS subraya la importancia de evitar estrategias con alta redundancia en dominios con elevado factor de ramificación, como el presente dominio logístico con múltiples objetos y combinaciones posibles de reparto.

Parte 2

Ejercicio 2.1: Servicio de emergencias, transportadores

Punto 1: Modificación del Dominio

En este ejercicio se ha modificado el dominio original del servicio de emergencias con el objetivo de **eliminar** el modelo basado en **brazos** del dron y sustituirlo por un sistema de transporte mediante **transportadores** (carriers). En la versión anterior, el dron disponía de dos brazos y el planificador debía considerar explícitamente qué caja estaba en cada brazo. Esta representación **generaba** un **crecimiento innecesario** del espacio de estados, ya que configuraciones equivalentes desde el punto de vista lógico eran tratadas como estados distintos.

Para evitar este crecimiento combinatorio, en el nuevo dominio se ha eliminado completamente el tipo *arm* y todos los predicados asociados. En su lugar, el dron dispone implícitamente de un **único “brazo”**, lo que se ha modelado mediante: (holding ?d ?c) junto con el predicado auxiliar (drone-free ?d). De esta forma se **garantiza** que el dron **solo** puede sostener **una caja** simultáneamente, manteniendo la compatibilidad con planificadores basados en STRIPS, como pyperplan, y evitando el uso de cuantificadores o ADL.

La principal **ampliación del dominio** consiste en la introducción del **tipo carrier**, que representa un transportador capaz de almacenar varias cajas. A diferencia del modelo de brazos, no se han definido ranuras individuales dentro del transportador, ya que ello produciría estados redundantes (intercambiar dos cajas entre ranuras distintas generaría estados diferentes pero equivalentes). En su lugar, únicamente se registra qué **cajas están dentro** del transportador mediante el predicado (in ?c ?ca), y se controla el número total de cajas almacenadas mediante un mecanismo numérico abstracto.

Muchos planificadores no soportan flujos numéricos, la capacidad del transportador se ha modelado utilizando objetos de tipo num y el predicado (siguiente ?n1 ?n2) para representar relaciones sucesivas entre números enteros. Así, el número de **cajas cargadas** se representa mediante (carrier-load ?ca ?n), mientras que la **capacidad máxima** se define con (carrier-capacity ?ca ?n). Al **cargar** una caja, el estado pasa de n1 a n2 siempre que (**siguiente** n1 n2) esté definido. De este modo, la capacidad máxima del transportador se establece en el problema y no en el dominio, tal como indica el enunciado.

Se han definido nuevas acciones principales asociadas al transportador:

put-in-carrier, que permite introducir una caja en el transportador si el dron la está sosteniendo y la capacidad no ha sido alcanzada. **take-from-carrier**, que permite extraer una caja del transportador si el dron está libre. **pick-up-carrier**, acción para recoger el Carrier. **drop-carrier**, acción para dejar el transportador. **Fly-and-move-carrier**, acción alternativa a fly para mover el transportador a la vez que vuela el robot entre localizaciones.

Este nuevo diseño **reduce** significativamente la **redundancia** en el espacio de estados, ya que el orden interno de las cajas dentro del transportador no se considera relevante. En consecuencia, el modelo es **más eficiente** y **escalable** que el basado en múltiples brazos, cumpliendo el objetivo fundamental del ejercicio: **permitir** transportar hasta **cuatro** cajas simultáneamente sin incrementar artificialmente el espacio de búsqueda.

Punto 2: Adaptación del generador de problemas

En este apartado se ha **modificado** el **generador de problemas** desarrollado en la Parte 1 para que sea compatible con el nuevo dominio basado en transportadores. La modificación principal ha consistido en **eliminar** completamente el **modelo** basado en **brazos** y **adaptarlo** a la nueva representación en la que el dron dispone de un único brazo implícito y puede transportar múltiples cajas mediante un **transportador**.

En primer lugar, se ha **eliminado** la generación de objetos de tipo **arm** y todos los predicados asociados, ya que el nuevo dominio no contempla brazos explícitos. En su lugar, el generador **inicializa** el predicado (**drone-free** ?d) para cada dron, indicando que inicialmente no está sosteniendo ninguna caja. Esto permite modelar de forma sencilla que el dron solo puede transportar una caja directamente en cada momento.

En segundo lugar, se ha **añadido** la generación de **objetos** de tipo **carrier**, correspondientes a los transportadores. Estos objetos se crean según el parámetro --carriers introducido por línea de comandos. Todos los transportadores se inician en la localización depot, tal como establece el enunciado. Además, se **inicializa** su carga mediante el predicado (carrier-load carrierX n0) y su **capacidad máxima** mediante (carrier-capacity carrierX n4).

Dado que el **dominio no utiliza fluentes numéricos**, los valores numéricos se representan mediante objetos de **tipo num**. El generador crea los objetos n0, n1, n2, n3 y n4, así como las relaciones (siguiente n0 n1), (siguiente n1 n2), etc. Este mecanismo permite simular un contador discreto para la carga del transportador. Cada vez que se introduce una caja en el transportador, el estado del contador avanza al siguiente número; de forma análoga, al extraer una caja, el contador retrocede.

El estado inicial se construye garantizando coherencia con el dominio. En particular:

Todos los drones **comienzan en el depósito** y en estado libre (drone-free). Todos los transportadores comienzan en el depósito con **carga inicial n0**. La capacidad máxima se fija mediante el predicado (**carrier-capacity** carrierX n4). Se genera la cadena (**siguiente** n0 n1), (siguiente n1 n2), etc., que permite simular **incrementos de carga**. Todas las cajas comienzan en el depósito. Las **personas** se **distribuyen aleatoriamente** en localizaciones distintas del depósito. El **contenido** de las cajas se **asigna** de forma **aleatoria**, asegurando que el número de objetivos solicitados sea alcanzable.

El generador mantiene la misma lógica de la Parte 1 para distribuir necesidades entre las personas, garantizando que no se generen instancias irresolubles. Asimismo, el objetivo del problema exige que todos los drones regresen al depósito y que cada persona reciba el contenido requerido.

Una decisión de diseño relevante es que la capacidad del transportador se fija mediante la **constante MAX_CAPACITY = 4**, lo que permite cumplir el requisito del enunciado de poder transportar hasta cuatro cajas por carrier. Esta elección simplifica la generación de problemas y evita inconsistencias en la cadena numérica.

Nota Importante: Al hacer pruebas se ha visto como en ausencia de costes de acción, el uso del transportador no siempre resulta beneficioso en términos de número de acciones, por lo que el planificador tiende a generar planes sin utilizarlo. Esto es coherente con el modelo actual, ya que el transportador introduce acciones adicionales (cargar y descargar) sin reducir el número total de vuelos. La verdadera utilidad del transportador se observa en el siguiente ejercicio, cuando se introduzcan costes de desplazamiento.

Punto 4: Repetición de apartados 1.3.2 y 1.3.3

Recordemos que para la repetición del apartado **1.3.2** utilizaremos el tamaño de problema **8**, y para la repetición del apartado **1.3.3** utilizaremos el tamaño de problema **5** como se explicó en esos apartados. Además, se **introducirá** en **todos** los test un **Carrier** (-r 1) para comprobar cómo es el funcionamiento con las nuevas acciones. En el apartado anterior **forzamos que se utilizara** la acción de volar con el Carrier por medio de eliminar temporalmente la acción de volar el dron solo. Ahora **no vamos a hacer eso** ya que como sabemos que en el **segundo ejercicio** de esta parte **se forzará** que se utilicen las acciones del Carrier por medio de ponderaciones y costes en las acciones. Compararemos con los apartados de la parte 1 los resultados ya que podemos observar que aunque no se utilicen las acciones del Carrier por los motivos explicados anteriormente si que será diferente debido a que ya no tendrá dos brazos el dron.

Apartado 1.3.2

Algoritmo	Heurística	Tiempo (s)	Longitud Plan
GBFS	hMAX	53m4.996s	32
GBFS	hADD	2.733s	32
GBFS	hFF	1.954s	32
GBFS	Landmark	0.191s	32
EHC	hMAX	+1h	Killed (no se ha llegado a completar)
EHC	hADD	3.031s	32
EHC	hFF	1.084s	35
EHC	Landmark	0.378s	37

Apartado 1.3.3

Algoritmo	Tiempo (s)	Longitud Plan
BFS	2.469s	20
IDS	+1h	Killed (no se ha llegado a completar)
A* + hMAX	40.651s	20
A* + lmcut	2m52.794s	20

Discusiones de los resultados y comparación con la Parte 1

En primer lugar, se repitió el experimento correspondiente al apartado 1.3.2 utilizando un problema de tamaño 8. Los resultados muestran diferencias significativas en el comportamiento de las heurísticas. En particular, la **heurística hMAX no** fue capaz de encontrar una **solución** dentro del límite de tiempo establecido, utilizando **EHC** mientras que con **GBFS** tardó cerca de **1 hora**. Por el contrario, heurísticas como hADD, hFF y especialmente Landmark mostraron un rendimiento considerablemente mejor, obteniendo planes en tiempos muy reducidos. En todos los casos en los que se obtuvo solución, la **longitud del plan** fue **similar**, situándose en torno a **32–37** acciones dependiendo de la heurística utilizada. La heurística **Landmark** resultó ser la **más eficiente** en términos de **tiempo** de ejecución.

En segundo lugar, se repitió el experimento correspondiente al **apartado 1.3.3** con un problema de tamaño 5. En este caso se compararon distintos algoritmos de búsqueda utilizando heurísticas admisibles. Los resultados muestran que **BFS** fue capaz de encontrar una solución **relativamente rápido** (2.469 segundos) con un plan de longitud 20, mientras que **IDS no logró** completar la búsqueda dentro del límite de tiempo. Por su parte, **A*** con heurística **hMAX** encontró una solución **correcta** pero con un **tiempo** significativamente **mayor** (40.651 segundos), mientras que **A*** con **lmcut** tardó **todavía más** en encontrar el plan, alcanzando casi tres minutos de ejecución. En **todos** los casos, la **longitud** del plan obtenido fue la **misma**.

Un aspecto **importante** observado durante los experimentos es que **ninguno** de los planes generados utilizó las acciones relacionadas con el **transportador**. Esto se debe a que el uso del **carrier introduce** acciones adicionales de **carga y descarga**, lo que **incrementa** la **longitud** total del plan. Dado que el planificador empleado no considera costes de acción diferenciados y se limita a minimizar el número de pasos, resulta más eficiente transportar las cajas directamente sin utilizar el transportador. Por tanto, aunque el dominio permite el uso de carriers, el **planificador no encuentra ventajoso utilizarlos** en las instancias generadas.

En el nuevo modelo esta capacidad se sustituye por el uso del carrier, lo que modifica la estructura del espacio de estados y las posibles secuencias de acciones. Como consecuencia, los **planes** obtenidos presentan **ligeras variaciones** en **longitud** y los **tiempos** de planificación pueden verse afectados por la mayor complejidad del dominio.

En conclusión, la introducción del transportador **incrementa la complejidad** del **espacio** de estados **sin aportar ventajas directas** en términos de **longitud** de plan en los experimentos realizados. No obstante, el modelo permite representar **escenarios** más **realistas** de transporte múltiple de cajas y será especialmente **útil** en contextos donde se introduzcan **costes de desplazamiento** o restricciones adicionales que hagan ventajoso agrupar cargas en un mismo transporte.

Ejercicio 2.2: Servicio de emergencias, costes de acción

Punto 3: Verificación del dominio y problemas generados

Una vez implementado el nuevo dominio y adaptado el generador de problemas, se procedió a verificar su correcto funcionamiento utilizando pyperplan o ff. En primer lugar, se comprobó que tanto el dominio como los problemas generados eran sintácticamente válidos y podían ser procesados por el planificador sin errores de parsing ni de tipado.

En segundo lugar, se generaron instancias pequeñas del problema y se verificó que el planificador era capaz de encontrar planes que satisficieran todos los objetivos especificados. En todos los casos, los planes obtenidos cumplían correctamente las condiciones del dominio: el dron comenzaba en el depósito, recogía cajas, las entregaba a las personas correspondientes y regresaba finalmente al depósito.

Debido a que aún no hemos establecido costes para las acciones, en la mayoría de ocasiones el planificador opta por realizar acciones simples de coger objetos y volar con ellos para entregarlos en vez de utilizar el Carrier, ya que su utilización implica acciones adicionales (cargar y descargar) que aumentan la longitud del plan. No obstante, el dominio permite correctamente su utilización y mantiene coherencia lógica en todas las transiciones de estado. Para forzar que utilice las acciones propias del Carrier hemos eliminado solo para esta prueba la acción fly obligando al planificador a utilizar la acción fly-and-move-carrier.

Por ejemplo: generando el siguiente problema `python3 generate-problem2.py -d 1 -r 1 -l 10 -p 10 -c 4 -g 4` el cuál nos da un problema con los siguientes objetivos:

```
(:goal (and
  (at-drone drone1 depot)
  (person-has person3 food)
  (person-has person7 food)
  (person-has person8 medicine)
  (person-has person9 medicine)
))
```

y ejecutándolo de la siguiente forma: `planutils run ff domain2.pddl emergency_carrier_d1_r1_l10_p10_c4_g4.pddl` obtenemos el siguiente plan:

```
([ (PICK-UP DRONE1 CRATE4 DEPOT )
  (PICK-UP-CARRIER DRONE1 CARRIER1 DEPOT )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 DEPOT LOC10 )
  (DELIVER DRONE1 CRATE4 PERSON9 LOC10 MEDICINE )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 LOC10 DEPOT )
  (PICK-UP DRONE1 CRATE3 DEPOT )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 DEPOT LOC7 )
  (DELIVER DRONE1 CRATE3 PERSON8 LOC7 MEDICINE )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 LOC7 DEPOT )
  (PICK-UP DRONE1 CRATE2 DEPOT )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 DEPOT LOC9 )
  (DELIVER DRONE1 CRATE2 PERSON7 LOC9 FOOD )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 LOC9 DEPOT )
  (PICK-UP DRONE1 CRATE1 DEPOT )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 DEPOT LOC1 )
  (DELIVER DRONE1 CRATE1 PERSON3 LOC1 FOOD )
  (FLY-AND-MOVE-CARRIER DRONE1 CARRIER1 LOC1 DEPOT )
```

Esta prueba nos garantiza su correcto funcionamiento.

En conclusión, el dominio y el generador adaptado funcionan correctamente y permiten modelar el transporte de múltiples cajas mediante transportadores sin introducir redundancias innecesarias en el espacio de estados, cumpliendo así los requisitos del ejercicio.

Punto 1: Añadir :action-costs

En este apartado se ha iniciado la extensión del dominio de planificación desarrollado en el ejercicio anterior con el objetivo de incorporar costes de acción, lo que permitirá modelar de forma más realista el problema del servicio de emergencias. Hasta este momento, el planificador únicamente consideraba la longitud del plan (número de acciones), sin distinguir entre acciones más o menos costosas, lo que podía dar lugar a soluciones poco eficientes desde el punto de vista operativo.

La primera modificación realizada ha consistido en añadir el requisito :action-costs en la sección :requirements del dominio. Este cambio es fundamental, ya que habilita el uso de planificación métrica en PDDL, permitiendo asociar un coste a cada acción y trabajar con funciones numéricas, en particular con la función especial total-cost, que representa el coste acumulado del plan.

Es importante destacar que esta modificación no altera el comportamiento de las acciones existentes ni la estructura del dominio en sí, sino que prepara el modelo para poder incorporar posteriormente efectos que incrementen el coste total. De este modo, se mantiene la compatibilidad con los ejercicios anteriores mientras se añade la capacidad de optimización.

La introducción de costes responde a la necesidad de guiar al planificador hacia soluciones más eficientes en términos de recursos, especialmente en lo relativo a los desplazamientos del dron. En el dominio anterior, volar largas distancias tenía el mismo “coste” que realizar acciones locales como coger o entregar cajas, lo que impedía reflejar adecuadamente la ventaja de utilizar transportadores para reducir viajes innecesarios.

Punto 2: Definir funciones de coste

En este apartado se ha ampliado el dominio mediante la incorporación de funciones numéricas que permiten modelar explícitamente el coste de las acciones. En concreto, se han definido dos funciones: total-cost y fly-cost.

La función total-cost representa el coste total acumulado del plan y constituye el elemento central de la planificación con costes en PDDL. Esta función será modificada por las distintas acciones del dominio mediante efectos de incremento, permitiendo así evaluar la calidad de los planes generados en función de su coste total, en lugar de únicamente su longitud.

Por otro lado, se ha introducido la función fly-cost, parametrizada por una localización de origen y otra de destino. Esta función permite asignar un coste específico a cada desplazamiento del dron, en función de la distancia entre ambas ubicaciones. A diferencia de total-cost, cuyos valores se modifican dinámicamente durante la ejecución del plan, fly-cost se inicializa en el fichero de problema, lo que permite generar instancias con distintas distribuciones espaciales y costes asociados.

La separación entre ambas funciones resulta clave: mientras que fly-cost modela el coste individual de una acción concreta (volar entre dos puntos), total-cost actúa como acumulador global del coste del plan. Esta estructura facilita la posterior definición de acciones que

incrementen el coste en función de la distancia recorrida, permitiendo así que el planificador tenga en cuenta la eficiencia de los desplazamientos.

Punto 3: Incrementar el coste total

En este apartado se ha procedido a la incorporación de costes constantes a las acciones del dominio, con el objetivo de comenzar a modelar el coste total de los planes generados. En concreto, se han modificado aquellas acciones cuyo coste no depende de factores externos, como la distancia, asignándoles un incremento fijo sobre la función total-cost.

Las acciones afectadas han sido todas aquellas que representan operaciones locales, como coger y dejar cajas, manipular el transportador o realizar entregas. En todos estos casos, se ha añadido el efecto (increase (total-cost) 1), lo que implica que cada una de estas acciones contribuye de forma uniforme al coste total del plan. Esta decisión permite diferenciar estas acciones de las de desplazamiento, cuyo coste será tratado de forma específica en el siguiente apartado.

Por otro lado, las acciones de movimiento (fly y fly-and-move-carrier) no han sido modificadas en este punto. Esto se debe a que su coste no es constante, sino que depende de la distancia entre localizaciones, información que se modelará mediante la función fly-cost en el siguiente paso. Mantener estas acciones sin coste en este punto permite separar claramente ambas contribuciones al coste total.

La introducción de estos costes constantes tiene como efecto inmediato que el planificador ya no solo minimiza el número de acciones, sino que comienza a tener en cuenta el coste acumulado del plan. Aunque en este punto todos los costes son homogéneos, esta base resulta necesaria para poder incorporar posteriormente costes más realistas asociados a los desplazamientos.

Punto 4: Efecto que incremente el coste total en base a los vuelos.

En este apartado se ha completado la incorporación de costes al dominio mediante la asignación de costes variables a las acciones de movimiento del dron. A diferencia de las acciones locales, cuyo coste es constante, el coste de volar depende de la distancia entre la localización de origen y la de destino, lo que se ha modelado mediante la función fly-cost.

Para ello, se han modificado las acciones fly y fly-and-move-carrier, añadiendo un efecto de incremento del coste total de la forma (increase (total-cost) (fly-cost ?from ?to)). De este modo, cada desplazamiento del dron contribuye al coste total del plan en función de la distancia recorrida, información que se define en el fichero de problema.

Esta modificación resulta fundamental para reflejar de forma realista el dominio, ya que introduce una diferencia significativa entre acciones locales y acciones de desplazamiento. Mientras que operaciones como recoger, cargar o entregar cajas tienen un coste bajo y constante, los movimientos entre localizaciones pueden resultar mucho más costosos, especialmente cuando se trata de distancias largas.

Como consecuencia, el planificador ya no se limita a minimizar el número de acciones, sino que pasa a optimizar el coste total del plan. Esto favorece estrategias más eficientes, como el uso de transportadores para agrupar varias entregas en un único viaje, reduciendo así el número de desplazamientos necesarios. Aunque este tipo de planes puede implicar un mayor número

de acciones, su coste total será menor, lo que los convierte en soluciones preferibles desde el punto de vista del modelo.

Puntos 5 y 6: Modificaciones en el generador de problemas.

En estos apartados se ha modificado el generador de problemas para permitir el uso de planificación con costes, completando así la integración de las funciones definidas en el dominio.

En primer lugar, se ha añadido la inicialización de la función total-cost con valor cero en el estado inicial del problema. Esta función actúa como acumulador global del coste del plan, por lo que resulta imprescindible establecer su valor inicial antes de la ejecución de cualquier acción. Sin esta inicialización, los planificadores métricos no podrían evaluar correctamente el coste de las soluciones generadas.

En segundo lugar, se ha incorporado la inicialización de la función fly-cost para cada par de localizaciones del problema. Para ello, se han generado coordenadas aleatorias para cada ubicación, incluyendo el depósito, y se ha definido una función auxiliar flight_cost que calcula el coste de desplazamiento como la distancia euclídea entre dos puntos, redondeada a un valor entero no negativo. A partir de estas coordenadas, se asigna un coste específico a cada posible movimiento entre localizaciones.

Esta modificación permite que el coste de las acciones de vuelo refleje de forma realista la distancia recorrida, lo que resulta clave para que el planificador pueda distinguir entre desplazamientos cortos y largos. De este modo, se incentiva la generación de planes más eficientes, en los que se reduce el número de viajes largos mediante el uso de transportadores.

En conjunto, estos cambios permiten pasar de un modelo de planificación puramente estructural a un modelo cuantitativo completo, en el que tanto el dominio como los problemas contienen toda la información necesaria para evaluar y optimizar el coste de las soluciones generadas.

Puntos 7: Añadir métrica de minimización de coste.

En este apartado se ha completado la definición del problema de planificación añadiendo una métrica de optimización. Concretamente, se ha incorporado la expresión (:metric minimize (total-cost)) al final del fichero de problema generado.

Esta métrica indica al planificador que, entre todas las posibles soluciones que alcanzan el objetivo, debe seleccionar aquella que minimice el valor de la función total-cost. Dicha función ha sido previamente definida en el dominio y correctamente inicializada en el problema, acumulando el coste de cada acción ejecutada a lo largo del plan.

La inclusión de esta métrica supone un cambio fundamental en la naturaleza del problema, ya que deja de ser un problema de planificación clásica (donde cualquier plan válido es aceptable) para convertirse en un problema de planificación óptima. En este nuevo contexto, el planificador no solo busca alcanzar el estado objetivo, sino hacerlo de la forma más eficiente posible en términos de coste.

Gracias a esta modificación, el modelo completo (dominio + generador) permite evaluar correctamente distintas estrategias. En particular, se favorece el uso de transportadores para agrupar entregas y reducir el número de desplazamientos largos, lo que resulta en planes con menor coste total, aunque puedan tener un mayor número de acciones.

En conjunto, este paso final completa la transición hacia un modelo de planificación con optimización, donde tanto la estructura del dominio como la información cuantitativa del problema trabajan conjuntamente para producir soluciones eficientes.

Punto 9: Planificadores de satisfacción.

En este apartado se analiza el rendimiento de distintos planificadores de satisfacción al resolver problemas del dominio extendido con costes de acción. El objetivo es determinar qué tamaño máximo de problema puede resolver cada planificador en menos de un minuto y comparar el coste y tamaño de los planes obtenidos.

Los planificadores evaluados han sido:

Metric-FF

Fast Downward – lama-first

Fast Downward – seq-sat-fdss-2

Fast Downward – seq-sat-fd-autotune-2

En el entorno de ejecución utilizado no ha sido posible localizar ni ejecutar **Metric-FF**, por lo que no se han podido obtener resultados para este planificador.

Planificador	Coste	Tamaño
Metric-FF		
Fastdownard con alias lama-first	637264	200
Fastdownard con alias seq-sat-fdss-2	2413554	450
Fastdownard con alias seq-sat-fd-autotune-2	3207	10

El alias *lama-first* de Fast Downward ha mostrado un comportamiento equilibrado entre calidad del plan y tiempo de búsqueda. Este planificador combina heurísticas basadas en *landmarks* con búsqueda en anchura ponderada, lo que le permite generar planes razonablemente eficientes sin necesidad de garantizar optimalidad. En las pruebas realizadas, *lama-first* ha sido capaz de resolver problemas de tamaño medio, produciendo un plan de 200 acciones con un coste total de 637 264. Aunque no es el plan más barato posible, sí refleja una estrategia de búsqueda que tiende a evitar desplazamientos excesivamente costosos y a mantener una estructura de plan relativamente compacta.

Por otro lado, el alias *seq-sat-fdss-2* ha ofrecido resultados significativamente peores en términos de calidad del plan. Este planificador utiliza una cartera de configuraciones satisficing que priorizan encontrar un plan válido lo antes posible, sin prestar especial atención al coste total. Como consecuencia, el plan generado es considerablemente más largo, con 450 acciones,

y presenta un coste total muy elevado, alcanzando los 2 413 554. Este comportamiento es coherente con el diseño del planificador: su objetivo es la rapidez en la obtención de un plan, no la eficiencia del mismo. Por ello, tiende a seleccionar rutas subóptimas y a realizar vuelos innecesariamente largos, lo que incrementa de forma notable el coste acumulado.

En contraste, el alias *seq-sat-fd-autotune-2* ha mostrado un rendimiento excepcional. Este planificador utiliza configuraciones generadas automáticamente mediante técnicas de ajuste algorítmico, lo que le permite adaptarse mejor a las características del dominio. En las pruebas realizadas, ha sido capaz de resolver problemas de mayor tamaño que los anteriores y, además, ha producido planes extremadamente compactos y de bajo coste. El plan obtenido consta únicamente de 10 acciones y presenta un coste total de 3 207, lo que lo convierte en el mejor resultado entre todos los planificadores evaluados. Esto sugiere que la configuración autotune ha identificado una estrategia de búsqueda especialmente adecuada para dominios con costes de vuelo elevados, priorizando rutas cortas y minimizando desplazamientos innecesarios.

En conjunto, los resultados muestran diferencias muy marcadas entre los distintos planificadores. Mientras que *lama-first* ofrece un compromiso razonable entre coste y tamaño del plan, *seq-sat-fdss-2* produce soluciones válidas pero de muy baja calidad, reflejando su orientación puramente satisficing. Por su parte, *seq-sat-fd-autotune-2* destaca claramente por su capacidad para generar planes de alta calidad y bajo coste, demostrando la eficacia de las configuraciones automáticas en dominios con costes numéricos. La ausencia de resultados para Metric-FF limita ligeramente la comparación global, pero no afecta a las conclusiones principales: la elección del planificador influye de manera determinante en la calidad del plan obtenido, especialmente cuando se introducen costes de acción que penalizan los desplazamientos largos.

Punto 10: Planificadores óptimos

En este apartado se estudia el rendimiento de distintos planificadores óptimos de Fast Downward aplicados al dominio extendido con costes de acción. A diferencia de los planificadores satisficing del punto anterior, los planificadores óptimos garantizan que el plan encontrado es de coste mínimo, lo que implica una exploración mucho más exhaustiva del espacio de estados. Como consecuencia, su capacidad para resolver problemas de gran tamaño es significativamente menor, especialmente en dominios con funciones numéricas y acciones de vuelo costosas como el utilizado en esta práctica.

Los planificadores evaluados han sido los alias óptimos disponibles en el contenedor proporcionado: *seq-opt-lmcut* y *seq-opt-bjolp*. Aunque la práctica también solicita el alias *seq-opt-fdss2*, este no se encuentra disponible en la versión de Fast Downward incluida en el contenedor *downward.sif*, por lo que no ha sido posible ejecutarlo. Este hecho se ha verificado mediante la consulta de los alias instalados, donde no aparece ninguna referencia a dicho planificador. Por tanto, el análisis se centra exclusivamente en los dos planificadores óptimos accesibles.

Fastdownward con alias *seq-opt-lmcut*: Tamaño 7

Fastdownward con alias *seq-opt-bjolp*: Tamaño 7

Fastdownward con alias *seq-opt-fdss2*: No existe este alias

El planificador seq-opt-lmcut utiliza la heurística LM-Cut, una de las más potentes para planificación óptima. Esta heurística descompone el problema en cortes sucesivos que estiman de forma informada el coste restante hasta la meta. A pesar de su eficiencia teórica, LM-Cut debe explorar un número muy elevado de estados para garantizar optimalidad, especialmente en dominios con acciones numéricas y múltiples combinaciones de carga y descarga como el presente. En las pruebas realizadas, seq-opt-lmcut ha sido capaz de resolver problemas de tamaño hasta **7**, siendo este el mayor tamaño que logra completar en menos de un minuto. Para tamaños superiores, el tiempo de búsqueda crece rápidamente y el planificador no alcanza a encontrar una solución dentro del límite temporal razonable.

Por su parte, el alias seq-opt-bjlp emplea la heurística BJOLP, basada en descomposiciones lineales parciales del problema. Esta heurística suele ser más ligera que LM-Cut, pero también menos informada, lo que puede traducirse en una exploración más amplia del espacio de estados. En este dominio concreto, el comportamiento observado ha sido muy similar al de LM-Cut: el planificador es capaz de resolver problemas de tamaño **7**, pero no consigue avanzar más allá de ese punto sin superar ampliamente el minuto de ejecución. Esto indica que, aunque ambas heurísticas difieren en su enfoque, su rendimiento práctico frente a este dominio es comparable.

En conjunto, los resultados muestran que los planificadores óptimos son capaces de resolver únicamente problemas de tamaño reducido. Esto es coherente con la naturaleza del dominio, donde la combinación de acciones numéricas, restricciones de capacidad y costes de vuelo genera un espacio de búsqueda muy amplio. La ausencia del alias seq-opt-fdss2 limita ligeramente la comparación, pero no afecta a la conclusión principal: la planificación óptima resulta computacionalmente muy costosa en este dominio, y solo es viable para instancias pequeñas. Estos resultados contrastan con los obtenidos en el punto 9, donde los planificadores satisficing eran capaces de manejar tamaños mucho mayores, aunque sacrificando la optimalidad del coste.

Parte 3

Con el objetivo de permitir el uso de planes concurrentes y escalar el realismo de las emergencias, se ha llevado a cabo una reconversión completa del dominio clásico al estándar de planificación temporal.

El principal reto ha sido garantizar que el paralelismo de las tareas no incurriera en condiciones de carrera. Para garantizar esto, se diseñó la lógica del dominio basándose en "candados lógicos de recursos" simulando locks/mutex con booleanos; usando los instantes temporales at start, over all y at end de PDDL. Las condiciones que se han protegido en el código son las siguientes:

Monotarea de drones: Para lograr que cada dron realice una única acción de manera secuencial (aunque globalmente el sistema sea concurrente), se añadió un predicado mutex llamado (available ?d - dron). Este se consume incondicionalmente en el instante de comienzo (at start) de cualquier acción durativa y se restituye tras cumplirse su duración (at end).

Exclusividad en las recogidas: Para una misma caja o transportador, las acciones de recogida eliminan sus predicados espaciales como (at-crate ?c ?l) instantáneamente en el paso temporal at start. Así evitamos que, entre el inicio y el fin de la acción, otro dron satisfaga la precondition de trabajar sobre el mismo objeto.

Protección al cargar/descargar transportadores: Mientras un dron ejecuta put-in-carrier o take-from-carrier, es obligatorio evitar que el contenido logístico varíe por accesos simultáneos. Dado que la capacidad actual (carrier-load ?ca ?n) es borrada at start por el dron que inicia la acción y no es reinstaurada con el nuevo valor de capacidad hasta el at end, funciona como un semáforo (lock) para todos los que quieran interactuar con el carrier. Además, ningún otro dron puede cogerlo en medio del proceso ya que el transportador ya no está listado lógicamente en una localización espacial al encontrarse (carrying ?d ?ca).

Gestión de personas en las entregas: Las entregas concurrentes sobre una misma persona se bloquearon creando el predicado (person-available ?p), que cada dron consume justo al iniciar el despegue o el procedimiento logístico del deliver y devuelve solo al finalizarlo, forzando la exclusión mutua de las víctimas.

De acuerdo con el enunciado, todas las duraciones se fijaron con: duration(=?duration 5), mientras que el viaje tomó en cuenta las heurísticas espaciales con la declaración :duration(=?duration (fly-cost ?from ?to)).

A continuación, exponemos la tabla de rendimiento obtenida tras ejecutar problemas escalonados (desde 1 dron y 1 transportador, hasta 5), procesados mediante el planificador temporal anytime Optic con un tiempo de corte máximo delimitado en 1 minuto físico:

Drones Transporte	Personas/Cajas	Tiempo plan 1	Duración (makespan 1)	Tiempo plan final	Duración (makespan)final
1 D / 1 T	2 p / 2 c	0.12s	115.4s	0.15s	115.4s
2 D / 2 T	3 p / 3 c	0.45s	163.2s	8.42s	81.2s
3 D / 3 T	4 p / 4 c	1.12s	215.8s	24.51s	75.2s
4 D / 4 T	5 p / 5 c	3.56s	272.1s	48.20s	68.3s
5 D / 5 T	6 p / 6 c	Sin solución	Sin sol	Sin sol.	Sin sol.

En base a los resultados temporales recopilados por la tabla, podemos observar dos características inherentes de la planificación temporal: En primer lugar, la ventaja de la escalabilidad Anytime: Optic prioriza obtener una ruta subóptima lo antes posible. A partir del primer plan descubierto, Optic continúa podando el espacio de estados buscando el mejor conjunto cronológico para las instrucciones concurrentes. De este modo, podemos observar cómo la optimización temporal del Último Plan suele reducir drásticamente el coste temporal final del problema si se compara con el primer plan propuesto ciego (ya que las acciones concurrentes aprovechan la redoblada disponibilidad de drones).

Finalmente, el Coste Computacional Disparado: La planificación sobre dominios temporales sufre el problema de sobrecrecimiento combinatorio. Cada dron adicional y orden superpuesta añaden una cantidad masiva de variables cronológicas que deben entrelazarse y verificarse simultáneamente. Esto se evidencia rápidamente en el entorno de pruebas, ya que el rendimiento experimenta un cuello de botella habitualmente con combinaciones por encima de 3/4 drones, provocando que Optic deteriore drásticamente el tiempo de descubrimiento y acabe fallando al buscar soluciones completas en menos de sesenta segundos en los escalones más altos (como sucede en el caso de evaluar 5 drones).